



Programovacie jazyky

Funkcie fold, fold', fold1, scan, unfold a typová trieda Foldable

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

5. apríla 2018

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Celozrnné programovanie

Celozrnné (wholemeal) programovanie

znamená myslieť vo veľkom (Ralf Hinze), t.j. namiesto:

- cesty vidieť celý graf
- konkrétneho riešenia vyvinúť rámec pre riešenie problémov
- s elementami pracovať s celým zoznamom

```
1 int acc = 0;  
2 for(int i = 0; i < lst.length; i++) {  
3     acc = acc + lst[i] * 3;  
4 }
```

```
1 let acc = sum $ map (*3) lst
```

Pomáha predchádzať „indexitíde“. (Richard Bird)

Implementácia sum, concat a map

```
1 sum :: Num a => [a] -> a
2 sum [] = 0
3 sum (x:xs) = x + sum xs
```

```
1 concat :: [[a]] -> [a]
2 concat [] = []
3 concat (x:xs) = x ++ concat xs
```

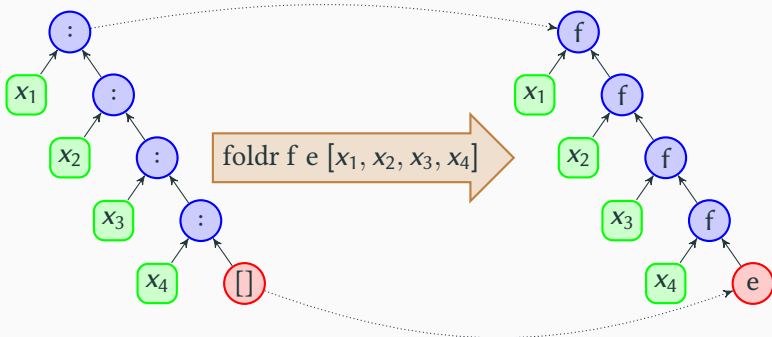
```
1 map :: (a -> b) -> [a] -> [b]
2 map _ [] = []
3 map g (x:xs) = [g x] ++ map g xs
```

- výsledná hodnota **e** pre prázdny zoznam
- kombinačná funkcia **f** pre zloženie prvej hodnoty zo zoznamu s výsledkom funkcie pre zvyšok zoznamu

Funkcia foldr

foldr f e

```
1 foldr :: (a->b->b) -> b -> [a] -> b  
2 foldr _ e [] = e  
3 foldr f e (x:xs) = x 'f' foldr f e xs
```

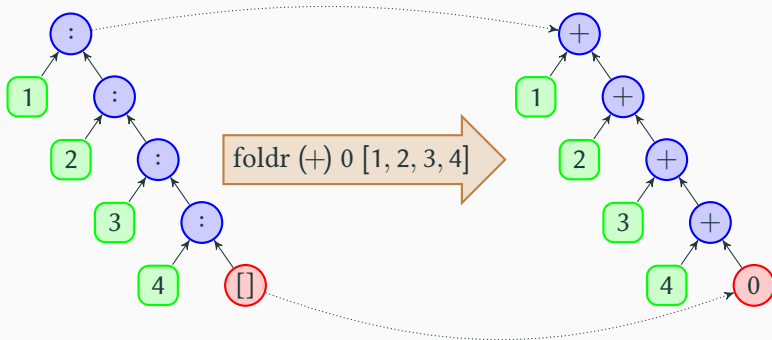


$$\text{foldr } (\oplus) \text{ e } [x_1, \dots, x_{n-1}, x_n] = x_1 \oplus (\dots (x_{n-1} \oplus (x_n \oplus e)) \dots)$$

sum pomocou foldr

```
1 foldr (+) 0 [] = 0
```

```
2 foldr (+) 0 (x:xs) = x + foldr (+) 0 xs
```



foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\lambda x \ xs \rightarrow f \ x \ : \ xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \ x \ n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

foldr – rôzne príklady

- `sum = foldr (+) 0`
- `product = foldr (*) 1`
- `concat = foldr (++) []`
- `and = foldr (&&) True`
- `or = foldr (||) False`

- `map f = foldr ((:) . f) []`
 - $((:) . f) \equiv (\backslash x \text{ } xs \rightarrow f \text{ } x : xs)$
- `filter p = foldr (\x xs -> if p x then x:xs else xs) []`

- `reverse = foldr post [] where post x xs = xs ++ [x]`
- `length = foldr inc 0 where inc x n = n + 1`
 - $inc \text{ } x \text{ } n = n + 1 \equiv inc = \text{const } (+1)$

HLint a foldr

HLint môže pomôcť odhaliť miesta, kde sa dá použiť foldr:

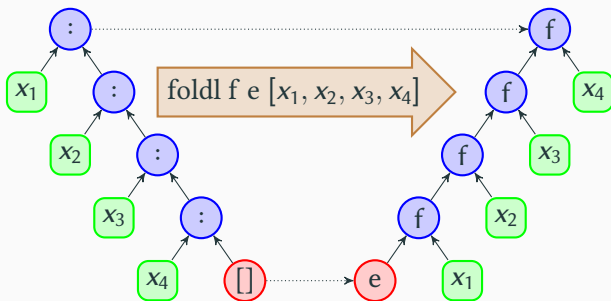
```
1 Suggestion: Use foldr
2 Found:
3   sum [] = 0
4   sum (x : xs) = x + sum xs
5 Why not:
6   sum xs = foldr (+) 0 xs
```

```
1 Suggestion: Use foldr
2 Found:
3   concat [] = []
4   concat (x : xs) = x ++ concat xs
5 Why not:
6   concat xs = foldr (++) [] xs
```

Funkcia foldl

foldl f e

```
1 foldl :: (b->a->b) -> b -> [a] -> b
2 foldl _ e [] = e
3 foldl f e (x:xs) = foldl f (f e x) xs
```



$$\text{foldl } (\otimes) e [x_1, x_2, \dots, x_n] = (\dots ((e \otimes x_1) \otimes x_2) \dots) \otimes x_n$$

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (**) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (**) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (**) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*) 0 where n ** x = 10*n + x`

foldl – příklady

- `sum = foldl (+) 0`
- `product = foldl (*) 1`
- `concat = foldl (++) []`
- `and = foldl (&&) True`
- `reverse = foldl (flip (:)) []`
- `length = foldl inc 0 where inc n x = n + 1`
 - `inc n x = n + 1  $\equiv$  inc = const . (+1)`
- `fromDigits :: Num a => [a] -> a`
`fromDigits = foldl (*+) 0 where n *+ x = 10*n + x`

HLint a foldl

HLint môže pomôcť odhaliť miesta, kde sa dá použiť foldl:

```
1 sum :: Num a => [a] -> a
2 sum list = go 0 list where
3   go a [] = a
4   go a (x:xs) = go (a + x) xs
```

```
1 Warning: Eta reduce
2 Found:
3   sum list = go 0 list
4 Why not:
5   sum = go 0
6
7 Suggestion: Use foldl
8 Found:
9   go a [] = a
10  go a (x : xs) = go (a + x) xs
11 Why not:
12  go a xs = foldl (+) a xs
```

Vety o foldr a foldl

1. veta o dualite foldr a foldl

Definícia

Trojica $M = (S, \otimes, e)$ je **monoid** práve vtedy, keď:

- $S = \{x_1, x_2, \dots\}$ je neprázdna množina,
- $\otimes : S \times S \rightarrow S$ je asociatívna operácia na množine S ,
- $e \in S$ je neutrálny prvok (zľava aj sprava) vzhľadom na $\otimes$.

Veta

Ak $(S, \otimes, e)$ je monoid a $xs \in S^*$ (konečný zoznam), tak:

$$\text{foldr } (\otimes) e xs = \text{foldl } (\otimes) e xs.$$

Príklady

- `sum` = `foldl (+) 0` = `foldr (+) 0`
- `product` = `foldl (*) 1` = `foldr (*) 1`
- `concat` = `foldl (++) []` = `foldr (++) []`
- `and` = `foldl (&&) True` = `foldr (&&) True`

2. veta o dualite foldr a foldl

Veta

Nech sú dané operácie $\oplus : A \times B \rightarrow B$, $\otimes : B \times A \rightarrow B$ a prvok $e \in B$.

Ak pre všetky prvky $x, x' \in A$ a $y \in B$ platí:

- $x \oplus (y \otimes x') = (x \oplus y) \otimes x'$
- $x \oplus e = e \otimes x$ (nemusi byť $= x$)

tak potom pre každý konečný zoznam $xs \in A^*$ platí:

$$\text{foldr } (\oplus) e xs = \text{foldl } (\otimes) e xs.$$

Poznámka

1. veta o dualite je špeciálny prípad 2. vety o dualite (pre $\oplus = \otimes$).

Príklady

- `reverse = foldr (\x xs -> xs ++ [x]) []`
- `reverse = foldl (\xs x -> x : xs) []`
- podobne `length`

3. veta o dualite foldr a foldl

Veta

Nech je daná operácia $\oplus : A \times B \rightarrow B$ a prvok $y \in B$. Pre každý konečný zoznam $xs \in A^*$ platí:

$$\text{foldr } (\oplus) y xs = \text{foldl } (\text{flip } (\oplus)) y (\text{reverse } xs).$$

Príklad

`id xs =`

- `foldr (\x xs -> x : xs) [] xs`
- `foldl (\xs x -> x : xs) [] (reverse xs)`

Poznámka

Táto veta vlastne definuje *foldr* pomocou *foldl*:

```
1 foldr f e xs =  
2     foldl (flip f) e (reverse xs)
```

Identity pre foldr a foldl

Príklady ďalších identít pre `foldr` a `foldl`:

Veta

Ak $(S, \otimes, e)$ je monoid a xs a ys sú zoznamy prvkov z S , tak:

$$\text{foldr } (\otimes) e (xs \mathbin{++} ys) = (\text{foldr } (\otimes) e xs) \otimes (\text{foldr } (\otimes) e ys).$$

Veta

Pre ľubovoľné, vhodne definované, f a a platí:

$$\text{foldr } f a (xs \mathbin{++} ys) = \text{foldr } f (\text{foldr } f a ys) xs$$

$$\text{foldl } f a (xs \mathbin{++} ys) = \text{foldl } f (\text{foldl } f a xs) ys$$

Poznámka

V predošlých vetách zoznamy xs a ys nemusia byť konečné.

Funkcie fold1 a scan

foldr1, foldl1

- pokiaľ nevieme zvoliť e (napr. pri hľadaní maxima)
- nepracuje na prázdnom zozname

```
1 foldr1 :: (a -> a -> a) -> [a] -> a
2 foldr1 _ []      = error "empty list"
3 foldr1 _ [x]     = x
4 foldr1 f (x:xs) = x 'f' foldr1 f xs
```

```
1 foldl1 :: (a -> a -> a) -> [a] -> a
2 foldl1 _ []      = error "empty list"
3 foldl1 f (x:xs) = foldl f x xs
```

Príklad

```
1 listMax :: [Int] -> Int
2 listMax = foldl1 max
3
4 maxVal = listMax [4,1,8,4,6]
```

scanr, scanl, scanr1, scanl1

Funkcie `scan` sú ako zodpovedajúce funkcie `fold`, ale vracajú aj zoznam všetkých medzivýsledkov (akumulátorov).

- `head (scanr f e xs) ≡ foldr f e xs`
- `last (scanl f e xs) ≡ foldl f e xs`

Príklady

```
scanr (:) [] [1..4]
```

```
↪ [[1,2,3,4],[2,3,4],[3,4],[4],[]]
```

```
scanl (flip (:)) [] [1..4]
```

```
↪ [[],[1],[2,1],[3,2,1],[4,3,2,1]]
```

```
scanl max 10 [2,7,4,15,12,18,7,3,20,19]
```

```
↪ [10,10,10,10,15,15,18,18,18,20,20]
```

```
scanl1 max [2,7,4,15,12,18,7,3,20,19]
```

```
↪ [2,7,7,15,15,18,18,18,20,20]
```

Funkcia unfold

```
1 unfoldr :: (b->Maybe (a, b))->b->[a]
```

Príklady

```
1 fromRevDigits :: [Integer] -> Integer
2 fromRevDigits = foldr f 0
3   where f c n = 10*n + c
```

`fromRevDigits [0,4,7,0,3,2]  $\rightsquigarrow$  230740`

```
1 toRevDigits :: Integer -> [Integer]
2 toRevDigits = unfoldr f' where
3   f' n = if n > 0
4     then Just (n `mod` 10, n `div` 10)
5     else Nothing
```

`toRevDigits 230740  $\rightsquigarrow$  [0,4,7,0,3,2]`

Výhody a nevýhody foldr a foldl

Rozdiel medzi foldr, foldl a fold'

```
import Data.List
```

- **Data.List.foldr** (+) 0 [1..1000000000]
 - ~> *** **Exception:** stack overflow
 - pretože **foldr** nepoužíva chvostovú rekurziu
- **Data.List.foldl** (+) 0 [1..1000000000]
 - ~> nedočkáte sa ☹️
 - odložený výpočet akumulátora vytvára obrovské množstvo nedokončených výpočtov na halde
 - nakoniec aj tak pretečie zásobník ☹️
 - pri vyhodnotení odložených výpočtov
- **Data.List.foldl'** (+) 0 [1..1000000000]
 - ~> 500000000500000000
 - skončí za pár sekúnd
 - chvostová rekurzia s priebežným vyhod. akumulátora

Kedy použiť foldr a kedy foldl'

foldr:

- pokiaľ f je nie je striktná (napr. `(&&)`, `(:)`)
 - je lenivá na svojom druhom argumente
- transformácia nekonečných zoznamov
 - `list = map (\x -> x /= 10000000) [1..]`
 - `foldr (&&) True list ~> False`

foldl':

- pokiaľ je funkcia f striktná (napr. `(+)`)
 - `Data.List.foldl' (+) 0 [1..1000000000]`

Typová trieda Foldable

Typová trieda Foldable

```
1 class Foldable (t :: * -> *) where
2   Data.Foldable.fold :: Monoid m => t m -> m
3   foldMap :: Monoid m => (a -> m) -> t a -> m
4   foldr :: (a -> b -> b) -> b -> t a -> b
5   Data.Foldable.foldr' :: (a -> b -> b) -> b -> t a -> b
6   foldl :: (b -> a -> b) -> b -> t a -> b
7   Data.Foldable.foldl' :: (b -> a -> b) -> b -> t a -> b
8   foldr1 :: (a -> a -> a) -> t a -> a
9   foldl1 :: (a -> a -> a) -> t a -> a
10  Data.Foldable.toList :: t a -> [a]
11  null :: t a -> Bool
12  length :: t a -> Int
13  elem :: Eq a => a -> t a -> Bool
14  maximum :: Ord a => t a -> a
15  minimum :: Ord a => t a -> a
16  sum :: Num a => t a -> a
17  product :: Num a => t a -> a
18  {-# MINIMAL foldMap | foldr #-}
```

Štandardne sú inštanciou tejto typovej triedy nasledovné typy:
[], Maybe, (Either a) a ((,) a).

Príklad inštancie Foldable pre strom

Algebraický dátový typ pre strom:

```
1 data Tree a = Empty
2             | Leaf a
3             | Node (Tree a) a (Tree a)
4             deriving (Show)
```

Príklad implementácie typovej triedy **Foldable** pre typový konštruktor **Tree** :: * -> *:

```
1 instance Foldable Tree where
2   foldr _ e Empty = e
3   foldr f e (Leaf x) = f x e
4   foldr f e (Node l v r) = foldr f (f v (foldr f e r)) l
5   -- stačí definovať foldr alebo foldMap
6   foldMap _ Empty = mempty
7   foldMap f (Leaf x) = f x
8   foldMap f (Node l v r) = foldMap f l `mappend`
9                           f v `mappend`
10                          foldMap f r
```

Automatické odvodenie inštancie Foldable pre strom

```
1 {-# LANGUAGE DeriveFoldable #-}  
2 data Tree a = Empty  
3             | Leaf a  
4             | Node (Tree a) a (Tree a)  
5             deriving (Show, Foldable)
```

```
1 tree = Node (Leaf 3)  
2         10  
3         (Node (Leaf 5)  
4             4  
5             Empty)
```

- `sum tree` $\rightsquigarrow$ 22
- `product tree` $\rightsquigarrow$ 600
- `foldMap (\x -> [x]) tree` $\rightsquigarrow$ [3,10,5,4]
- `Data.Foldable.toList tree` $\rightsquigarrow$ [3,10,5,4]